# `std::breakpoint`

René Ferdinand Rivera Morell, Isabella Muerte – grafikrobot@gmail.com – P2514R0, 2021-12-30

## Table of Contents



**Document number:**

ISO/IEC/JTC1/SC22/WG21/P2514R0

**Date:**

2021-12-30

**Audience:**

SG15, LEWG

**Reply-to:**

René Ferdinand Rivera Morell, Isabella Muerte, grafikrobot@gmail.com

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

## 1. Abstract

This paper proposes a new function, `std::breakpoint`, that causes a program to stop or "break" execution when it is being debugged to aid in software development.

This is a successor paper to P1279. [1]

## 2. Revision History

### 2.1. Revision 0 (January 2022)

Initial text based on P1279. [1]

## 3. Motivation

Setting breakpoints inside of a debugger can be difficult and confusing for newcomers to C++. Rather than having to learn C++, they have to learn a special syntax just to place a breakpoint in the exact spot they want, or rely on the interface of an IDE. At the end of the day, an average programmer just wants to place a breakpoint so that their program stops when under the watchful eye of a

debugger.
Having this facility also helps in advanced software development environments as it allows for runtime control of breakpoints beyond what might be available from a debugger. In particular to allow programmatic control on what runtime sensitive conditions to break into the debugger.

# 4. Design Decisions

The goal of the `std::breakpoint` function is to "break" when being debugged but to act as though it is a no-op when it is executing normally. This might seem difficult in practice, but nearly every platform and various debuggers supports something to this effect. However, some platforms have caveats that make implementing this "break when being debugged" behavior hard to implement correctly.

The `std::breakpoint` function is intended to go into a `<debugging>` header.

## 4.1. Impact On the Standard

This proposal adds a utility header and a single function the implementation of which is widely available across compilers and platforms.

# 5. Implementation Experience

In addition to the prototype implementation [2] there are the following, full or partial, equivalent implementations:

- The Microsoft Visual compiler provides a `__debugbreak` function that implements an unconditional break. [3]
- GNU Compiler Collection provides a `__builtin_trap` function that implements an unconditional break. [4]
- Clang provides a `__builtin_debugtrap` function that implements an unconditional break.
- The arm Keil armcc compiler provides a `__breakpoint` function that implements an unconditional break. [5]
- Unreal Engine 4 implements a similar facility as a macro.

# 6. Wording

Wording is relative to N4868 (https://wg21.link/N4868). [6]

## 6.1. Library

Add a new entry to General utilities library summary [utilities.summary] table.

| | | |
|---|---|---|
| [debugging] | Debugging | `<debugging>` |

Add section to General utilities library [utilities].

### 6.1.1. Debugging [debugging]

#### 6.1.1.1. In general [debugging.general]

Subclause [debugging] describes the debugging library that provides functionality to introspect and interact with a debugger that is executing and monitoring the running program.

#### 6.1.1.2. Header `<debugging>` synopsis [debugging.syn]

```C++
namespace std {
    // [debugging.utility], utility
    void breakpoint() noexcept;
}
```

#### 6.1.1.3. Utility [debugging.utility]

`void breakpoint() noexcept;`

*Remarks*: When this function is executed, it first must perform an implementation defined check to see if the program is currently running under a debugger. If it is, the program's execution is temporarily halted and execution is handed to the debugger until such a time as: the program is terminated by the debugger or, the debugger hands execution back to the program.

## 7. Acknowledgements

Thank you Isabella Muerte for the initial proposal from which this paper steals a good amount of the text.

---

1. P1279 std::breakpoint, *Isabella Muerte* 2018-10-05 (https://wg21.link/P1279)
2. Debugging prototype implementation (https://github.com/grafikrobot/debugging)
3. Microsoft compiler `__debugbreak` intrinsic (https://docs.microsoft.com/en-us/cpp/intrinsics/debugbreak)
4. GNU GCC Other Built-in Functions Provided by GCC (https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html)
5. armKEIL `__breakpoint` intrinsic (https://www.keil.com/support/man/docs/armcc/armcc_chr1359124993371.htm)
6. N4868 Working Draft, Standard for Programming Language C++ 2020-10-18 (https://wg21.link/N4868)